Projekt MPUM - Raport

Jakub Trela Karol Bielaszka

Nasz projekt znajduje się w repozytorium: github.com/kubat1234/calories-predictor.

Wstęp

W ramach naszego projektu pracujemy na zbiorze danych Food.com Recipes and Reviews dostępnym w serwisie Kaggle. Zawiera on przepisy kulinarne składające się z opisu dania, spisu składników, instrukcji przygotowania, wartości odżywczych (w tym liczby kalorii, tłuszczów, węglowodanów i białek), a także liczby porcji.
W naszych modelach jako cechy (argumenty wejściowe) rozważamy przede wszystkim same instrukcje przygotowania potrawy, jednak przetestujemy również warianty modeli rozszerzone o dodatkową cechę, jaką jest liczba porcji. Analizując instrukcje tekstowe, pomijamy wartości liczbowe, rozważamy wyłącznie słowa występujące w tekście. Zadaniem modeli jest predykcja liczby kalorii, tłuszczów, węglowodanów oraz białek.
Z uwagi na specyfikę problemu, gdzie dla modelu sformułowania typu „5g masła" oraz „100g masła" będą zinterpretowane dokładnie tak samo, nie spodziewamy się ekstremalnie wysokich wyników metryk jakościowych. Można jednak założyć, że część istotnych informacji uda się wywnioskować, ponieważ przepisy zamieszczane w Internecie są na ogół przygotowywane dla skończonej i powtarzalnej liczby osób (najczęściej 2–4). Z tego względu rozbieżność w rzeczywistych masach składników nie powinna być drastycznie wielka.

Analiza danych

W pliku `src/analyze_data.py` przeprowadzamy analizę danych. Nasz dataset składał się z wielu kolumn, my skupiliśmy się na przewidywaniu kalorii, węglowodanów, białka i tłuszczu na podstawie tekstowych instrukcji oraz liczby porcji (tylko niektóre modele).

Przykładowy wiersz z datasetu:

```
Porcje: 4 Kalorie: 203.90 kcal Tłuszcz: 7.30g Węglowodany: 31.90g Białko: 2.50g
```

```
Instrukcje: c("Pour cherry pie filling, cinnamon, cloves, and orange juice into a large pan.", Ądd cherry liqueur and bring to a boil, stirring constantly.", Ścoop ice cream into sundae dishes.", "Top with hot cherries.")
```

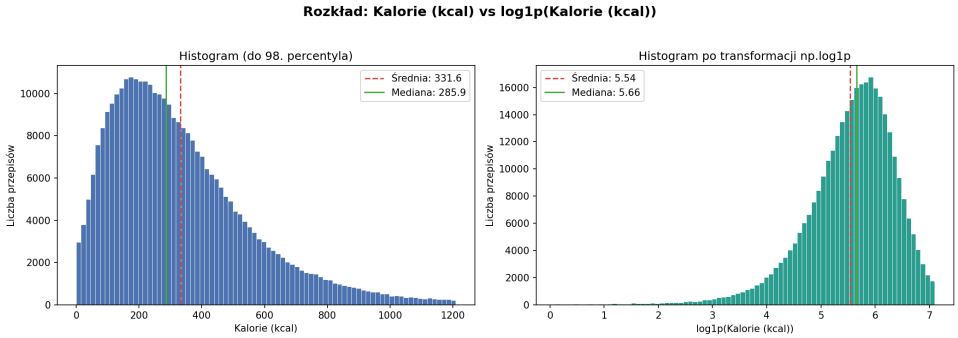
Przy wczytywaniu danych pomijałem te, które mają `NA` (brak danych) w co najmniej jednej z kolumn oraz te, które mają 0 we wszystkich kolumnach z wartościami liczbowymi (te wiersze wydawały się być wierszami z brakującymi danymi, ponieważ przepisy wyglądały

normalnie).

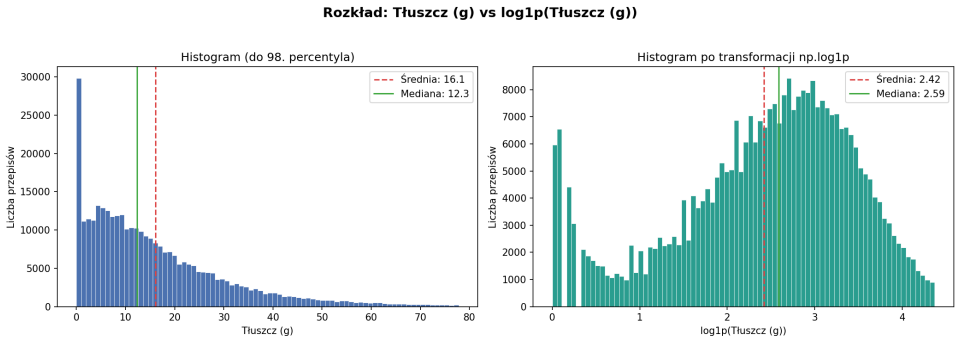
Statystyka	Kalorie (kcal)	Tłuszcz (g)	Węglowodany (g)	Białko (g)
Max	434 360.20	4 701.10	108 294.60	4 437.20
98 perc.	1 209.63	77.80	118.00	63.80
Średnia	371.82	18.76	34.99	15.96
Mediana	291.20	12.70	25.90	8.90
Odch. std.	878.20	31.94	193.74	24.61

Tablica 1: Statystyki opisowe wartości odżywczych (N = 337 685, śr. dł. instrukcji: 592.53 znaków)

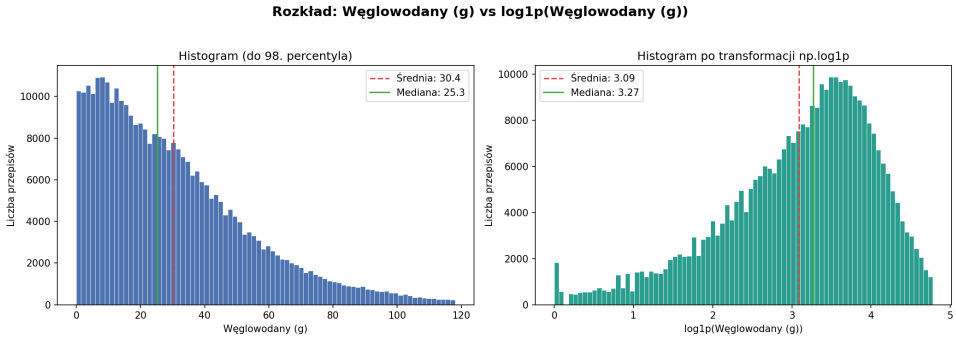
W datasecie występują wartości odstające, które mogą znacząco wpłynąć na wyniki analizy. W związku z tym zdecydowaliśmy się na obcięcie wartości do 98. percentyla. Poniżej znajdują się wykresy rozkładu dla każdej z analizowanych cech po obcięciu wartości odstających. Wykresy mają rozkłady skośne (im większe wartości, tym mniej danych), więc po prawej znajdują się wykresy po zlogarytmowaniu, które przypominają rozkład normalny, co jest korzystne dla wielu modeli.



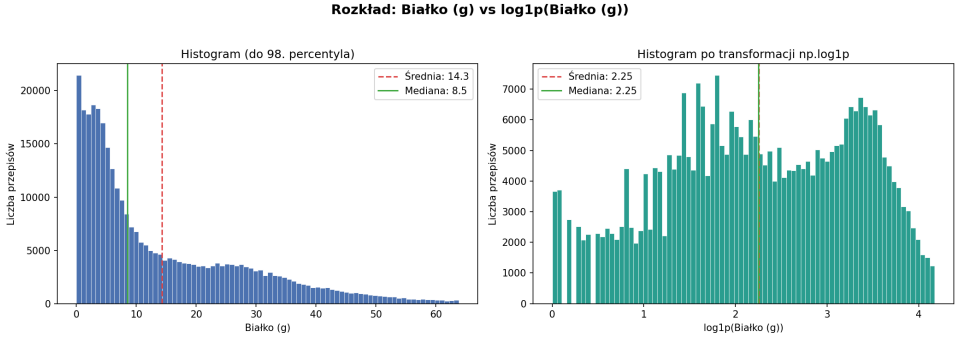
Rysunek 1: Rozkład kalorii w zbiorze danych.



Rysunek 2: Rozkład tłuszczu w zbiorze danych.



Rysunek 3: Rozkład węglowodanów w zbiorze danych.



Rysunek 4: Rozkład białka w zbiorze danych.

Ekstrakcja cech

W plikach w `src/get_data.py`, `src/features.py` oraz `src/features.py` wczytujemy dane, tokenizujemy tekst i wektoryzujemy.

Tokenizacja

Z tekstu usuwamy interpunkcję, liczby i znaki specjalne, a następnie dzielimy tekst na słowa.

Wektoryzacja

Testujemy różne metody wektoryzacji:

- Zwykłe zliczanie słów (Count Vectorizer)
  Polega na zliczaniu wystąpień poszczególnych słów w dokumencie.
- Term Frequency-Inverse Document Frequency (TF-IDF)
  Mierzy ważność słowa w dokumencie, uwzględniając jego częstość w całym korpusie. Term Frequency (TF) to liczba wystąpień słowa w dokumencie, a Inverse Document Frequency (IDF) to logarytmiczna odwrotność liczby przepisów zawierających to słowo.
  TF-IDF jest obliczane jako iloczyn TF i IDF.

- TF-IDF z cechami manualnymi
  Oprócz standardowego TF-IDF, dodajemy cechy takie jak liczba słów lub cechy dla różnych kategorii (np. fats and sweets, proteins, cooking methods heavy). Każda kategoria ma pewien zbiór słów kluczowych, których szukamy w tekście.

- Word2Vec
  Każde słowo jest reprezentowane jako wektor w przestrzeni wielowymiarowej, a dokument jest reprezentowany jako średnia wektorów słów w nim zawartych. Do wyliczenia wektorów trenujemy algorytm Word2Vec na naszym korpusie instrukcji.

W Count i TF-IDF odrzucamy słowa, które występują w małej liczbie przepisów, np. w mniej niż 3 przepisach, oraz takie, które występują w bardzo dużej liczbie przepisów, np. w więcej niż 90% przepisów. Wybieramy też stałą liczbę cech (np. 5000) i bierzemy tylko te najczęściej występujące (aby uniknąć zbyt dużej liczby cech, które mogą spowolnić działanie modelu).
W tych 2 metodach używamy też bigramów (par słów występujących obok siebie).

Porównanie metod wektoryzacji

W pliku `compare_vectorizers.py` porównujemy różne metody wektoryzacji.

Metody te porównujemy używając prostego modelu regresji liniowej z regularyzacją L2 (Ridge) i porównując średni błąd bezwzględny (MAE), pierwiastek średniego błędu kwadratowego (RMSE) i współczynnik determinacji (R^2).
(R^2 mówi nam, o ile model jest lepszy niż prosty model, który zawsze przewiduje średnią wartość. $R^2 = 0$ oznacza, że model jest tak dobry jak ten prosty model, a $R^2 = 1$ oznacza idealne dopasowanie do danych).

Ponieważ wykresy rozkładu cech są skośne, to modele przewidują logarytm wartości, a następnie bierzemy wykładnik z przewidywań, aby otrzymać przewidywane wartości.

Proste zliczanie radzi sobie bardzo słabo, nawet gorzej od modelu, który zawsze przewiduje średnią wartość. TF-IDF daje znaczną poprawę, a dodanie cech manualnych daje jeszcze niewielką poprawę. Word2Vec radzi sobie gorzej niż TF-IDF, co może być spowodowane tym, że średnia wektorów słów może nie być najlepszym sposobem reprezentacji całego przepisu.

Warto zauważyć, że przewidywanie ilości białka jest łatwiejsze niż przewidywanie kalorii oraz pozostałych makroskładników.

Założenia odnośnie modeli

Do treningu modeli wykorzystujemy zbiór składający się z $120\,000 - \epsilon$ wierszy, gdzie wartość ϵ wynika z procesu odfiltrowania niepoprawnych lub brakujących cech. Do testów rezerwujemy osobny podzbiór liczący $80\,000 - \epsilon$ wierszy. Przed podziałem dane zostały w pełni przetasowane.
Do generowania wykresów (zarówno krzywych uczenia, jak i analizy wpływu maksymalnej liczby cech wyekstrahowanych z tekstu) wykorzystujemy zbiór o wielkości $60\,000 - \epsilon$

Tablica 2: Wyniki ewaluacji modeli na zbiorze walidacyjnym

Metoda	Zmienna docelowa	MAE	RMSE	R^2
Count	Calories	144.968	220.498	0.0309
	Fat	9.711	19.886	-0.6010
	Carbohydrates	16.910	35.965	-1.0373
	Protein	8.221	15.138	-0.0421
TF-IDF	Calories	135.859	193.744	0.2518
	Fat	8.801	13.839	0.2246
	Carbohydrates	15.344	22.762	0.1840
	Protein	7.053	11.411	0.4079
TF-IDF + Manual	Calories	134.645	191.838	0.2664
	Fat	8.676	13.678	0.2426
	Carbohydrates	15.093	22.465	0.2051
	Protein	7.018	11.381	0.4110
Word2Vec	Calories	151.638	214.352	0.0842
	Fat	9.926	15.258	0.0574
	Carbohydrates	16.995	24.716	0.0378
	Protein	8.100	12.852	0.2489

wierszy, z którego 20% wydzielamy jako zbiór walidacyjny. Krzywe uczenia obliczamy odpowiednio dla wartości około (5%, 19%, 32%, 46%, 59% 72%, 86%, 100%) rozważanych danych, a krzywe ilości cech na wartościach (10, 50, 100, 300, 600, 1000, 1500, 2000, 4000) cech. Do wektoryzacji tekstu wykorzystujemy metodę TF-IDF – zarówno w wersji z biblioteki `scikit-learn`, jak i w ramach własnej implementacji. Wstępne testy wykazały, że podejście to radzi sobie znacznie lepiej niż alternatywne metody reprezentacji tekstowej. Rozważamy również wariant „manual TF-IDF". Główną metryką, na podstawie której oceniamy i porównujemy dopasowanie modeli, jest współczynnik determinacji R^2.

Rozważane modele

Dumb regressor

W pierwszej kolejności rozważyliśmy model referencyjny, który zawsze zwraca wartość średnią z danych treningowych. Pozwoli to ocenić realny zysk z zastosowania bardziej zaawansowanych algorytmów.

Wyniki modelu: tfidf_dumb

Model	Target	MAE	RMSE	R2
tfidf_dumb	calories	159.772	200.836	-0.000
tfidf_dumb	fat	10.667	13.780	-0.000
tfidf_dumb	carbohydrates	18.516	23.206	-0.000
tfidf_dumb	protein	11.177	13.627	-0.000

Widzimy, jak wyglądają błędy i dokładność R^2, która szczególnie będzie nas interesować.

Elastic net gradient descent regressor

Pierwszym naszym modelem jest regresja liniowa z regularyzacją Elastic Net, zaimplementowana przy pomocy algorytmu stochastycznego spadku gradientu. Z racji nieróżniczkowalności składnika L_1 w punkcie zero, w implementacji zastosowaliśmy technikę *soft thresholding*. Jeżeli nasz model przestaje się uczyć, wprowadziliśmy mechanizm dynamicznego zmniejszania współczynnika uczenia (*learning rate*) w celu precyzyjniejszego zlokalizowania minimum globalnego. Jeśli pomimo tego zabiegu wynik na zbiorze walidacyjnym nie ulega poprawie, proces uczenia jest przerywany.

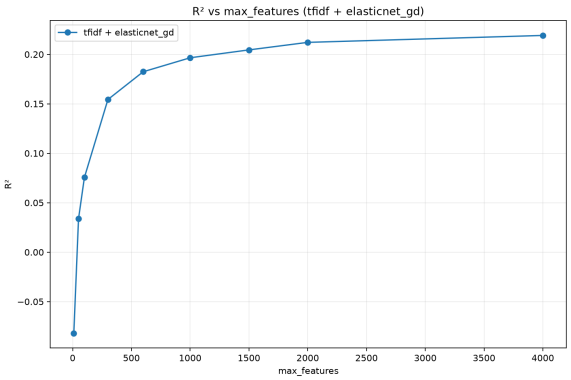
Wyniki modelu: tfidf_elasticnet_gd

Model	Target	MAE	RMSE	R2
tfidf_elasticnet_gd	calories	127.452	175.184	0.239
tfidf_elasticnet_gd	fat	8.154	12.188	0.218
tfidf_elasticnet_gd	carbohydrates	14.847	21.044	0.177
tfidf_elasticnet_gd	protein	6.674	10.442	0.413

Wyniki modelu: my_tfidf_elasticnet_gd

Model	Target	MAE	RMSE	R2
my_tfidf_elasticnet_gd	calories	126.936	174.598	0.244
my_tfidf_elasticnet_gd	fat	8.119	12.139	0.224
my_tfidf_elasticnet_gd	carbohydrates	14.720	20.946	0.185
my_tfidf_elasticnet_gd	protein	6.628	10.422	0.415

Widzimy, że model działa dość dobrze jak na problem, z jakim się mierzymy. Na naszym TF-IDF otrzymujemy nieco lepsze wyniki.



Zależność od ilości cech wybranych z tekstu wygląda dość spodziewanie. W praktyce musimy wybrać tylko kilka najważniejszych słów, aby określić kaloryczność.

Ridge

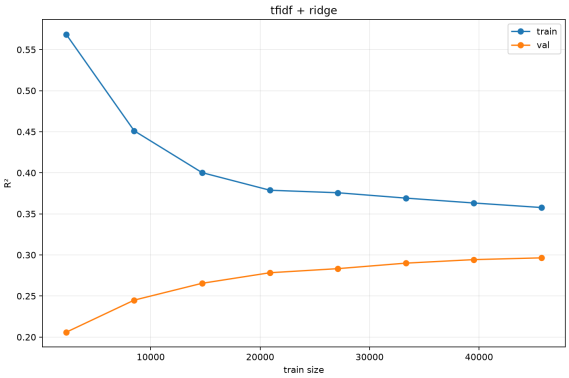
Dla porównania efektywności optymalizacji gradientowej wykorzystaliśmy model regresji grzbietowej z biblioteki `scikit-learn`.

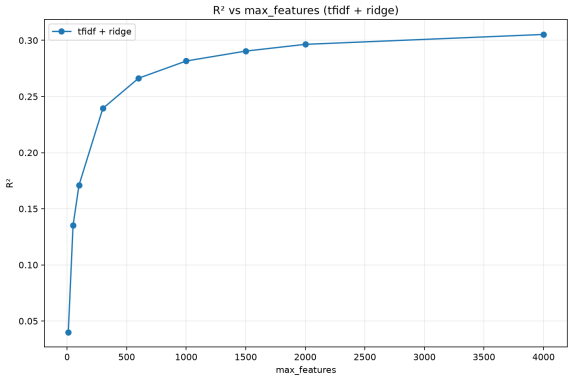
Wyniki modelu: tfidf_ridge

Model	Target	MAE	RMSE	R2
tfidf_ridge	calories	126.429	165.880	0.318
tfidf_ridge	fat	8.238	11.310	0.326
tfidf_ridge	carbohydrates	14.657	19.455	0.297
tfidf_ridge	protein	6.477	9.287	0.535

Wyniki modelu: my_tfidf_ridge

Model	Target	MAE	RMSE	R2
my_tfidf_ridge	calories	125.658	172.591	0.261
my_tfidf_ridge	fat	8.048	12.019	0.239
my_tfidf_ridge	carbohydrates	14.649	20.893	0.189
my_tfidf_ridge	protein	6.596	10.444	0.412





Widzimy, że model ze scikit poradził sobie lepiej, jednak gorzej współpracuje z naszym wektoryzerem. Krzywa zależności ilości cech wygląda niemal identycznie. Natomiast model na małych porcjach danych bardzo łatwo się przeucza, jednak przy nieco większej procji już jest w porządku.
Zaimplementowaliśmy również modele bazujące na ręcznie przygotowanym wektorze cech (manual TF-IDF), oraz uwzględniające dodatkowo informację o liczbie porcji, co miało na celu zmniejszenie błędu wynikającego ze skalowalności przepisów.

Wyniki modelu: manual_tfidf_ridge

Model	Target	MAE	RMSE	R2
manual_tfidf_ridge	calories	126.343	165.395	0.322
manual_tfidf_ridge	fat	8.227	11.285	0.329
manual_tfidf_ridge	carbohydrates	14.700	19.415	0.300
manual_tfidf_ridge	protein	6.472	9.283	0.536

Wyniki modelu: tfidf_ridge_servings

Model	Target	MAE	RMSE	R2
tfidf_ridge_servings	calories	121.797	168.569	0.295
tfidf_ridge_servings	fat	7.930	11.919	0.252
tfidf_ridge_servings	carbohydrates	14.463	20.635	0.209
tfidf_ridge_servings	protein	6.479	10.292	0.429

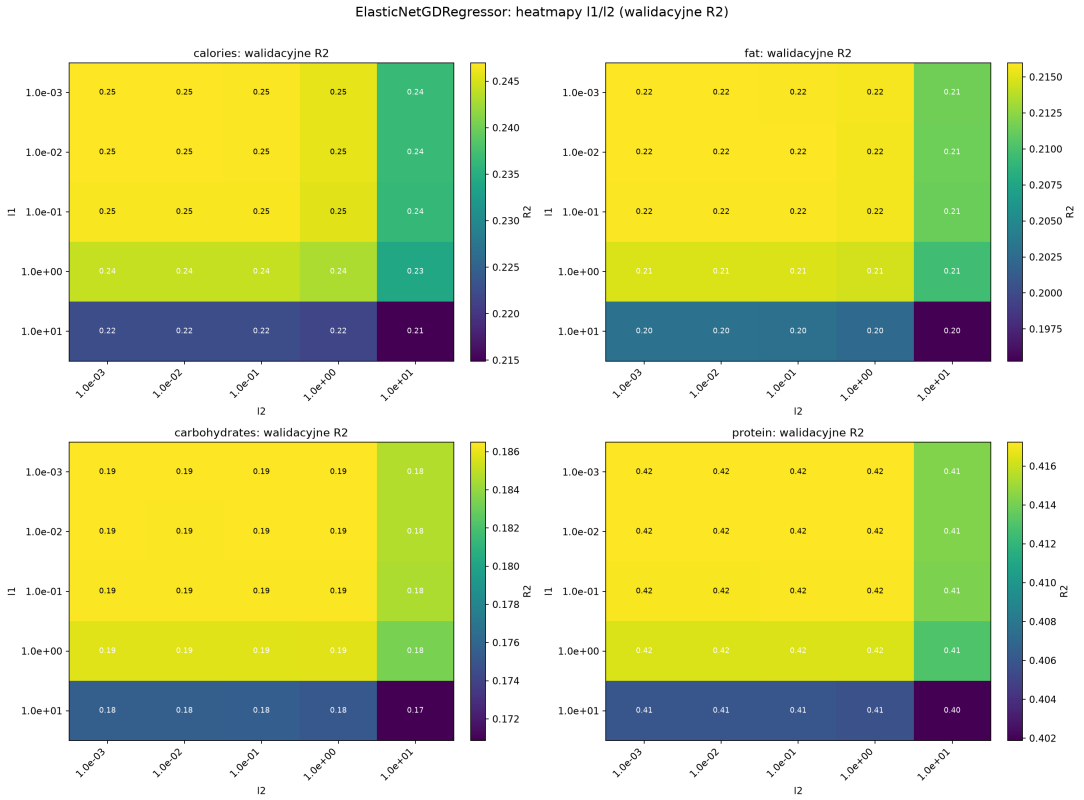
Wyniki modelu: manual_tfidf_ridge_servings

Model	Target	MAE	RMSE	R2
manual_tfidf_ridge_servings	calories	121.687	168.638	0.295
manual_tfidf_ridge_servings	fat	7.935	11.948	0.248
manual_tfidf_ridge_servings	carbohydrates	14.465	20.666	0.207
manual_tfidf_ridge_servings	protein	6.487	10.311	0.427

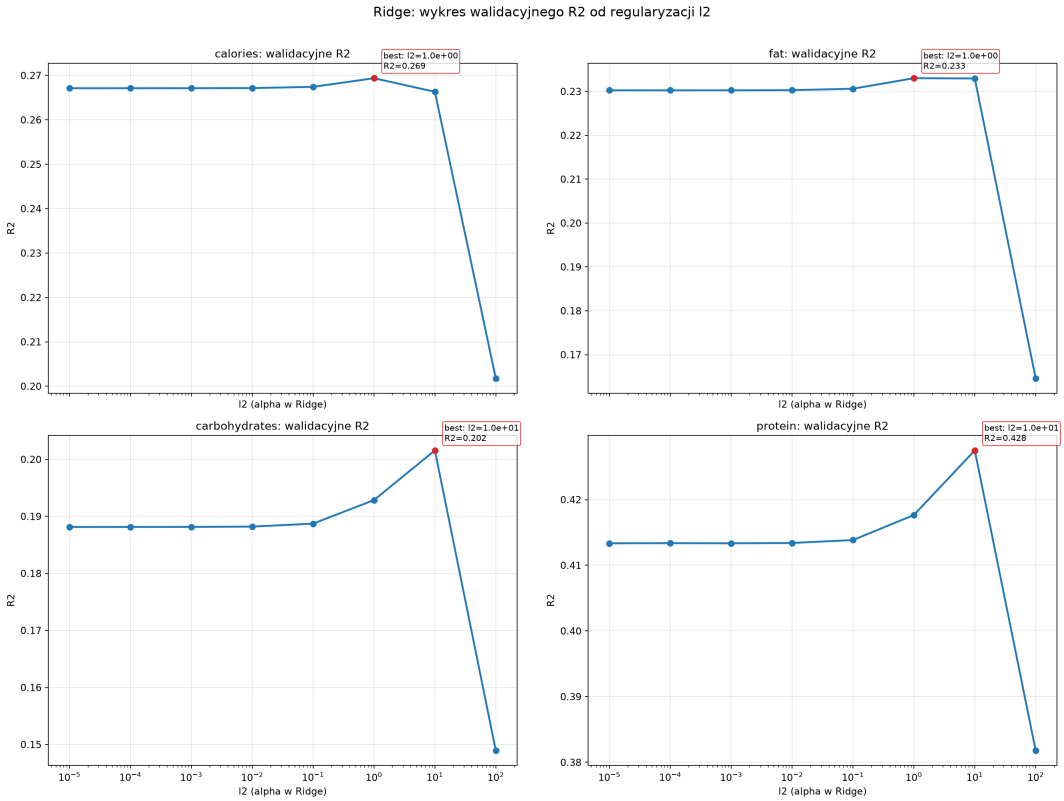
Możemy zauważyć, że dodanie manualnych cech poprawia model, jednak ilość porcji go osłabia.

Regresja z regularyzacją

W plikach `regularization_heatmap.py` oraz `regularization_plot.py` testujemy analityczną regresję liniową z regularyzacją L2 (Ridge) oraz regresję z siecią elastyczną (ElasticNet) z gradient descent. Poniżej znajdują się wyniki współczynnika determinacji (R^2) dla różnych wartości parametrów regularyzacji. (Im więcej, tym lepiej)



Rysunek 5: Współczynnik (R^2) dla regresji Elastic Net w zależności od parametrów regularyzacji l1 i l2.



Rysunek 6: Współczynnik (R^2) dla regresji Ridge w zależności od parametru regularyzacji l2.

W regresji Elastic Net najlepiej sprawdzają się małe wartości regularyzacji, a w regresji Ridge najlepiej sprawdzają się wartości 1 lub 10, w zależności od zmiennej docelowej.

Wyniki regresji Ridge są nieco lepsze niż Elastic Net, prawdopodobnie dlatego, że rozwiązanie analityczne jest dokładniejsze niż gradient descent, który może nie znaleźć najlepszego rozwiązania.

Neural net

Kolejnym rozważanym przez nas rozwiązaniem jest sieć neuronowa. Biorąc pod uwagę specyfikę oraz charakter zadania, struktura sieci nie musi być nadmiernie głęboka. Zdecydowaliśmy się na architekturę o dwóch warstwach ukrytych o wielkości odpowiednio 64 i 32 neuronów z funkcjami aktywacji ReLU. Model trenowaliśmy przez 20 epok z learning rate 0.1 oraz batchami wielkości 64. Parametry dobraliśmy eksperymentalnie.

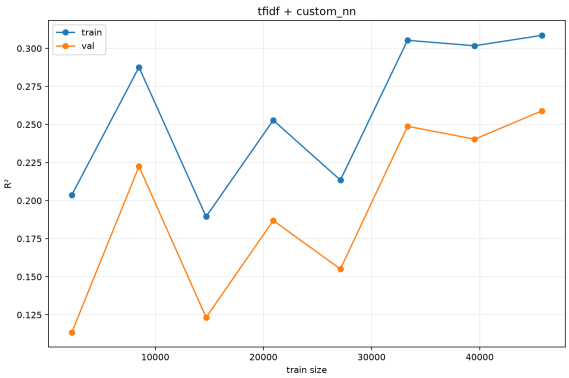
Wyniki modelu: tfidf_custom_nn

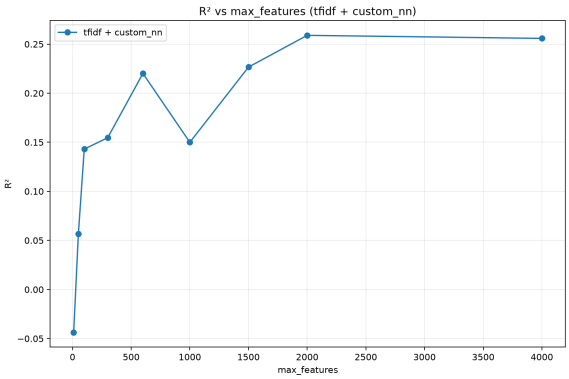
Model	Target	MAE	RMSE	R2
tfidf_custom_nn	calories	125.904	171.906	0.267
tfidf_custom_nn	fat	7.817	11.798	0.267
tfidf_custom_nn	carbohydrates	14.121	19.765	0.274
tfidf_custom_nn	protein	6.063	9.473	0.517

Wyniki modelu: my_tfidf_custom_nn

Model	Target	MAE	RMSE	R2
my_tfidf_custom_nn	calories	124.379	169.315	0.289
my_tfidf_custom_nn	fat	7.719	11.701	0.279
my_tfidf_custom_nn	carbohydrates	13.827	19.641	0.283
my_tfidf_custom_nn	protein	5.972	9.336	0.531

Wyniki są nieco lepsze od naszego modelu regresji liniowej, jednak i tak słabsze od gotowych modeli.





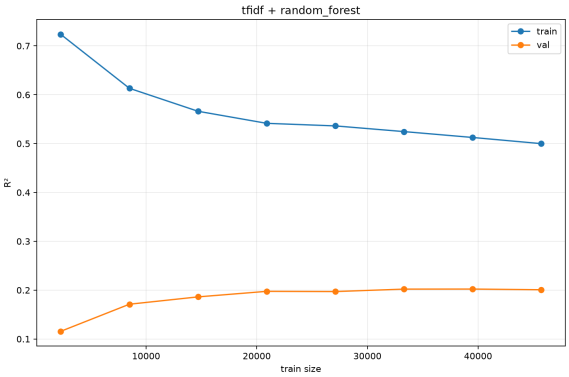
Analizując krzywą uczenia, możemy dojść do wniosku, że model wpada w lokalne minima. Próbowaliśmy to naprawić za pomocą optymalizacji Adam, jednak nie udało nam się znaleźć parametrów, dla których model zwróciłby dodatnie R^2. Wniosek jest taki, że zdecydowanie można poprawić ten model, ale sam wynik nie jest zły.

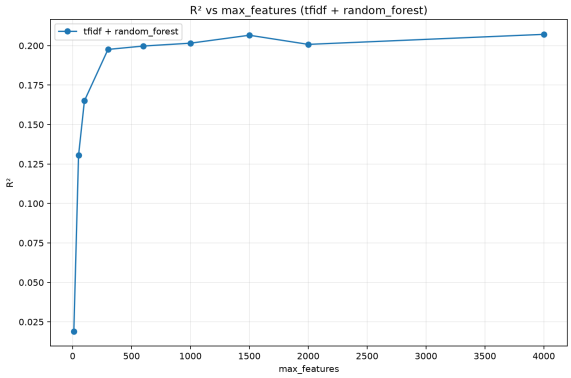
Random Forest

Model random forest trenowaliśmy z 10 drzewami o głębokości 15.

Wyniki modelu: my_tfidf_random_forest

Model	Target	MAE	RMSE	R2
my_tfidf_random_forest	calories	135.311	175.656	0.235
my_tfidf_random_forest	fat	9.065	12.230	0.212
my_tfidf_random_forest	carbohydrates	16.801	21.502	0.141
my_tfidf_random_forest	protein	7.674	10.646	0.390





Mimo tak małej liczby drzew, model uzyskuje dość sensowny wynik, ale widać na krzywych, że bardzo szybko się wypłaszczają, co może świadczyć o prostocie modelu.

Light gradient boosted machine

Jest to model, który korzysta z boostingu, gdzie jako słabe estymatory wykorzystuje drzewa decyzyjne. Z najważniejszych optymalizacji to ograniczenie nie na głębokość, lecz na liczbę liści w drzewie, dzielenie przedziału wartości ciągłych na bloki w preprocesingu, tak aby rozważać tylko ograniczoną ilość podziałów, pomijanie części obserwacji o małych wartościach gradientu, skupiając się jedynie na wartościach trudnych do sklasyfikowania.

W każdym momencie do drzewa dochodzi dokładnie jeden podział, który najbardziej poprawia wynik. Jest to podatne na przeuczenie, dlatego mamy stosunkowo niskie ograniczenie na ilość liści. Algorytm korzysta z wielu wątków podczas rozważania podziałów liści, przez co jest jeszcze szybszy. Dzięki temu mogliśmy wytrenować model z 1000 drzew. Każde drzewo miało 31 liści.

Wyniki modelu: my_tfidf_lgbm

Model	Target	MAE	RMSE	R2
my_tfidf_lgbm	calories	121.046	167.291	0.306
my_tfidf_lgbm	fat	7.571	11.451	0.309
my_tfidf_lgbm	carbohydrates	13.707	19.723	0.277
my_tfidf_lgbm	protein	6.022	9.515	0.512

Wyniki modelu: tfidf_lgbm

Model	Target	MAE	RMSE	R2
tfidf_lgbm	calories	119.878	166.361	0.314
tfidf_lgbm	fat	7.490	11.392	0.317
tfidf_lgbm	carbohydrates	13.664	19.698	0.279
tfidf_lgbm	protein	5.928	9.395	0.525

Wyniki modelu: manual_tfidf_lgbm

Model	Target	MAE	RMSE	R2
manual_tfidf_lgbm	calories	119.725	165.957	0.317
manual_tfidf_lgbm	fat	7.482	11.364	0.320
manual_tfidf_lgbm	carbohydrates	13.643	19.670	0.281
manual_tfidf_lgbm	protein	5.958	9.482	0.516

W tym przypadku nasz wektoryzer delikatnie pogarsza wynik, natomiast cechy manualne polepszają.

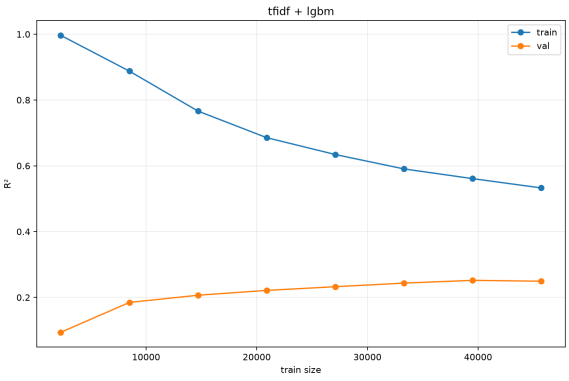
Wyniki modelu: tfidf_lgbm_servings

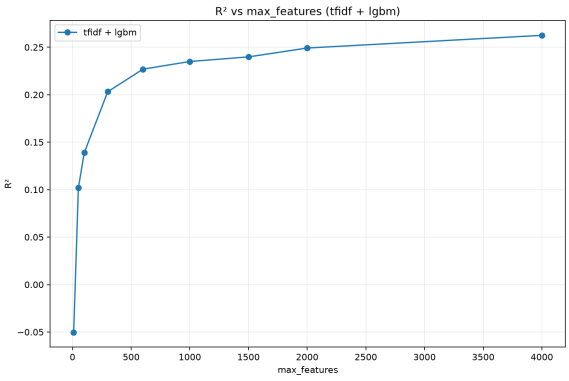
Model	Target	MAE	RMSE	R2
tfidf_lgbm_servings	calories	109.630	154.118	0.411
tfidf_lgbm_servings	fat	7.090	10.894	0.375
tfidf_lgbm_servings	carbohydrates	12.774	18.570	0.359
tfidf_lgbm_servings	protein	5.527	8.966	0.567

Wyniki modelu: manual_tfidf_lgbm_servings

Model	Target	MAE	RMSE	R2
manual_tfidf_lgbm_servings	calories	107.944	151.908	0.428
manual_tfidf_lgbm_servings	fat	7.005	10.779	0.388
manual_tfidf_lgbm_servings	carbohydrates	12.606	18.350	0.375
manual_tfidf_lgbm_servings	protein	5.439	8.836	0.580

Dodając do tego ilości porcji, uzyskujemy najlepszy wynik, jaki udało nam się uzyskać.





Analizując krzywą uczenia, możemy zobaczyć, że wypłaszcza się ona znacznie wolniej niż w przypadku krzywej modelu Ridge. Również krzywa ilości cech jest mniej wypłaszczona. Przypuszczalnie wynika to z tego, że model LGBM jest bardziej skomplikowany i uczy się wolniej, jednak jest w stanie nauczyć się więcej. Prawdopodobnie przy większych zasobach danych i mocy obliczeniowej bylibyśmy w stanie wytrenować go jeszcze lepiej.

Custom AdaBoost Regressor

Widząc wyniki LGBM, chcieliśmy napisać coś podobnego. Bez tylu optymalizacji, ale żeby było nasze. Udało nam się to napisać z ograniczeniem na głębokość. Jednak model z grubsza nie działa, a przynajmniej nie działa współpracując z mocą obliczeniową, jaką posiadamy. Próbując wytrenować model z 40 drzewami, zajęło to nam prawie 4.5h. A niestety model nic nie przewidywał. Dodanie większej ilości drzew mogłoby pomóc, jednak nie bylibyśmy w stanie tego wytrenować.

Sieć Neuronowa

W pliku `neural_network.py` testujemy sieć neuronową `MultiTargetRecipeRegressor` z biblioteki PyTorch, która jest w stanie przewidywać jednocześnie wszystkie 4 zmienne

docelowe. Sieć ta składa się z warstwy embedding (która zamienia słowa na wektory), average pooling (która uśrednia wektory słów, aby umożliwić trenowanie sieci na danych o różnej długości) oraz kilku warstw liniowych z funkcją aktywacji ReLU.

Aby zapobiec overfittingowi, używamy dropoutu (losowe wyłączanie części neuronów podczas treningu).

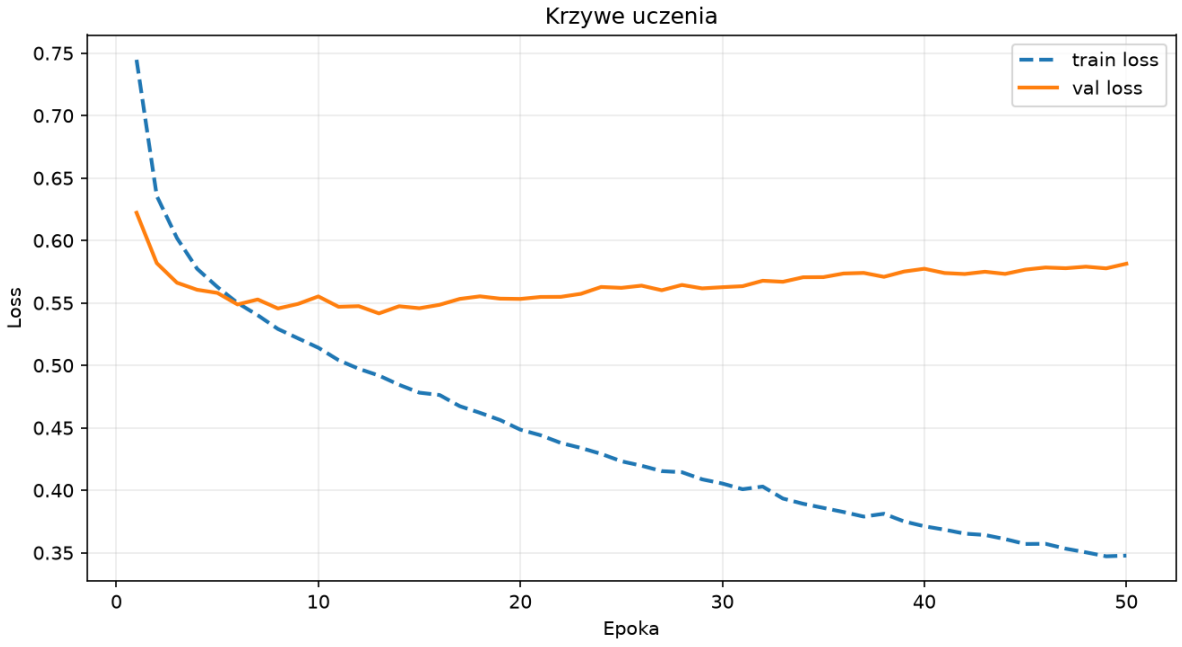
Używamy optymalizatora AdamW, który jest rozszerzeniem optymalizatora Adam o regularyzacje.

Poniżej tabelka z wynikami. Widać, że sieć neuronowa radzi sobie dosyć dobrze, najlepiej przewiduje ilość białka, tak jak inne modele.

Poniżej znajduje się też wykres strat w zależności od epoki. Widać, że po paru epokach loss na zbiorze walidacyjnym zaczyna rosnąć, co oznacza, że sieć zaczyna się przeuczać. Nawet wysoki współczynnik dropout i regularyzacje nie pomogły w pełni zapobiec overfittingowi.

Tablica 3: Wyniki metryk regresji dla poszczególnych zmiennych docelowych

Target	**MAE**	**MSE**	**RMSE**	**R2**
calories	113.9429	24199.7188	155.5626	0.4071
fat	7.7016	117.4648	10.8381	0.3874
carbohydrates	13.3620	338.4456	18.3969	0.3827
protein	5.7947	79.3103	8.9056	0.5736



Rysunek 7: Wykres strat dla sieci neuronowej w zależności od epoki.

Podsumowanie

- Na podstawie naszych obserwacji najlepszym wektoryzerem jest generalnie TF-IDF. Wyniki pomiędzy naszą implementacją, implementacją ze scikit-learn oraz połączenie z manualem dawały różne wyniki dla różnych modeli, zatem nie możemy określić, który z nich jest absolutnie najlepszy.
- Analizując proste modele, możemy zauważyć, że są one w stanie dość dobrze przewidzieć wynik (R^2 około 0.25). Jednak aby uzyskać lepsze wyniki, potrzebujemy dość złożonych modeli jak LGBM z 1000 drzew, lub dużej sieci neuronowej.
- Najlepszy uzyskany wynik to 0.43 dla kalorii i 0.58 dla białek, następny była sieć neuronowa z PyTorch z wynikami odpowiednio 0.41 i 0.57.
- Każdy model najlepiej radził sobie z szacowaniem ilości białek, prawdopodobnie dlatego, że białko pochodzi ze ściśle określonych produktów (np. kurczak albo mleko), a pozostałe rzeczy występują praktycznie wszędzie.
- Biorąc pod uwagę trudność przewidywania kalorii, uznajemy nasze wyniki za zadowalające.